**INSTITUTO DO EMPREGO E FORMAÇÃO PROFISSIONAL, I.P.**
CENTRO DE EMPREGO E FORMAÇÃO PROFISSIONAL DE BRAGA

EFA – PROGRAMADOR/A DE INFORMÁTICA

# UFCD 19 (0816) – Java Avançado

## Sistema de Avaliação de Conforto Térmico de Bovinos

(ConfortAnimal)

**Lucas Gonçalves, 08**
**Formador: Rui Boticas**
**IEFP Braga - Mazagão**

PESSOAS 2030 · PORTUGAL 2030 · Cofinanciado pela União Europeia
Os Fundos Europeus mais próximos de si.

# Índice

1. Introdução ao Projeto....2
2. O Problema Identificado....2
3. A Solução Desenvolvida....3
4. Funcionalidades da Aplicação....3
5. Entidades do Sistema....5
6. Estrutura Técnica....7
7. Conceitos de Programação Orientada a Objetos....8
8. Base de Dados....9
9. Dificuldades Encontradas....10
10. Conclusão....11
TABELAS NO MYSQL: ....12
ENTIDADES NO JAVA: ....12
Estrutura: ....12
Login:....13
Logout....13
Aba Animais:....13
Aba Ambientes: ....13
Aba Avaliações: ....14

# 1. Introdução ao Projeto

Boa tarde professor, Gostaria de apresentar o trabalho final que desenvolvi para a UFCD 19 - Java Avançado, que consistiu na criação de um sistema de avaliação de conforto térmico de bovinos, chamado ConfortAnimal. Este projeto foi uma grande oportunidade de aprendizagem para mim, pois consegui aplicar praticamente todos os conceitos que aprendemos durante o curso. Apesar de ser iniciante em programação, tentei criar algo que fosse não só funcional, mas também útil no mundo real. O sistema foi desenvolvido inteiramente em Java, utilizando a interface gráfica Swing, e conecta-se a uma base de dados MySQL remota. Escolhi este tema porque me interessei pela área da zootecnia e pela importância do bem-estar animal na produção agrícola.

# 2. O Problema Identificado

Durante a pesquisa para este projeto, descobri que o estresse térmico é um problema muito sério na pecuária. Quando os bovinos ficam expostos a temperaturas e humidade elevadas, isso afeta diretamente a produção de leite (podendo reduzir até 30%), causa problemas de saúde e diminui o bem-estar dos animais. Os produtores precisam de uma forma objetiva de medir se os animais estão em conforto térmico ou não. É aqui que entra o ITU - Índice de Temperatura e Umidade, uma métrica científica reconhecida internacionalmente que calcula o nível de conforto térmico com base na temperatura e humidade do ambiente.

A fórmula do ITU é: **ITU = 0.8 × T + (UR/100) × (T - 14.4) + 46.4**

Onde T é a temperatura em graus Celsius e UR é a umidade relativa do ar em percentagem. A classificação é feita da seguinte forma:

• ITU menor que 72: Animal em CONFORTO (sem stress)

• ITU entre 72 e 79: Animal em ALERTA (atenção necessária)

• ITU maior ou igual a 79: Animal em ESTRESSE (ação urgente)

# 3. A Solução Desenvolvida

Para resolver este problema, desenvolvi o ConfortAnimal, um sistema que automatiza o processo de avaliação do conforto térmico dos bovinos. O sistema permite que o produtor:

1. Cadastre todos os seus bovinos com informações como nome, raça, peso, idade e produção de leite

2. Registe as condições ambientais (temperatura e umidade) de diferentes locais da propriedade

3. Gere avaliações automáticas que calculam o ITU e classificam o estado de cada animal

4. Consulte o histórico de todas as avaliações realizadas

Desta forma, o produtor tem uma ferramenta objetiva para tomar decisões sobre quando precisa implementar medidas de conforto térmico, como ventiladores, sombreamento ou água fresca.

# 4. Funcionalidades da Aplicação

O sistema possui uma interface gráfica desenvolvida em Java Swing, organizada em três abas principais:

## 4.1. Aba Animal

Nesta aba, o utilizador pode registar novos bovinos no sistema. Os campos obrigatórios são:

• Nome do animal

• Raça (por exemplo: Holstein, Jersey, Girolando)

• Linhagem: LEITE (para vacas leiteiras), CARNE (para gado de corte) ou DUPLA_APTIDAO (misto)

• Peso em quilogramas (tem que ser maior que zero)

• Idade em meses (não pode ser negativa)

• Produção de leite em litros por dia (zero para gado de corte)

O sistema valida todos estes campos antes de permitir o registo. Por exemplo, não deixa inserir peso zero ou idade negativa. Também é possível listar todos os animais já cadastrados e apagar animais que já não existem na propriedade.

## 4.2. Aba Ambiente

Esta aba permite registar as condições climáticas de diferentes locais:

• Local (por exemplo: Estábulo Principal, Pasto Norte, Curral Externo)

• Temperatura em graus Celsius

• Umidade relativa do ar em percentagem

• Data e hora (registadas automaticamente pelo sistema)

Esta funcionalidade é importante porque diferentes áreas da propriedade podem ter condições climáticas diferentes (um estábulo coberto versus um pasto aberto, por exemplo).

### 4.3. Aba Avaliações

Esta é a aba mais importante do sistema! Aqui acontece a mágica da avaliação do conforto térmico. O utilizador seleciona um ambiente previamente cadastrado (com a temperatura e humidade já definidas), e o sistema:

1. Busca TODOS os bovinos registados no sistema

2. Para cada bovino, calcula o ITU usando a fórmula científica

3. Classifica o resultado como CONFORTO, ALERTA ou ESTRESSE

4. Grava todas as avaliações na base de dados

5. Mostra os resultados numa tabela clara e organizada

Por exemplo, se existem 5 bovinos cadastrados e o utilizador gera uma avaliação para o ambiente 'Estábulo Principal (28.5°C / 75%)', o sistema cria 5 avaliações diferentes, uma para cada animal, todas usando as mesmas condições ambientais mas guardando o resultado individual de cada um.

## 5. Entidades do Sistema

O sistema é composto por 5 entidades principais. Aprendi que as entidades são como as 'coisas' principais que o sistema precisa guardar informação. No meu caso:

### 5.1. Usuario

Esta entidade controla quem pode entrar no sistema. Tem os seguintes campos:

• id: número único de cada utilizador

• nomeUtilizador: o login para entrar no sistema

• senha: a palavra-passe

• nomeCompleto: nome completo da pessoa

• eAdmin: verdadeiro ou falso, indica se é administrador

No meu sistema tenho dois utilizadores de teste: 'admin' com senha 'admin' (administrador) e 'lucas' com senha 'lucas' (utilizador normal).

## 5.2. Animal e Bovino (Herança)

Aqui aprendi um conceito muito importante chamado HERANÇA. A classe Animal é a classe 'pai' que tem as propriedades básicas que todos os animais têm:

• id: identificador único

• nome: nome do animal

• peso: peso em quilogramas

• idade: idade em meses

A classe Bovino é a classe 'filha' que HERDA todas estas propriedades do Animal e adiciona as suas próprias:

• bovinoId: identificador único do bovino

• animalId: referência para o animal pai

• raca: raça do bovino

• linhagem: LEITE, CARNE ou DUPLA_APTIDAO

• producaoLeite: produção em litros por dia

No início tive dificuldade em entender este conceito, mas o professor explicou que é como uma árvore genealógica: o Bovino 'herda' características do Animal, mas tem as suas próprias características específicas também. Na base de dados, isto fica em duas tabelas separadas (animal e bovino) ligadas por uma chave estrangeira. Usei a propriedade PROTECTED nos campos do Animal para que o Bovino possa aceder a eles no método validar().

## 5.3. Ambiente

Guarda as condições climáticas de um local específico:

• id: identificador único

• temperatura: temperatura em graus Celsius

• umidade: humidade relativa em percentagem (0-100)

• local: nome do local (Estábulo, Pasto, etc.)

• dataRegisto: quando foi registado (NOW() do MySQL)

### 5.4. Avaliacao

Esta é a entidade que junta tudo! Cada avaliação liga um bovino específico com um ambiente específico:

• id: identificador único da avaliação

• bovinoId: qual bovino foi avaliado (chave estrangeira)

• ambienteId: qual ambiente foi usado (chave estrangeira)

• valorITU: o valor calculado do ITU

• resultado: CONFORTO, ALERTA ou ESTRESSE

• dataAvaliacao: quando foi feita a avaliação

• nomeAnimal: guardado para facilitar a exibição

• localAmbiente: guardado para facilitar a exibição

## 6. Estrutura Técnica

O professor ensinou-nos a organizar o código em 'camadas', e tentei seguir isso no meu projeto. O sistema está dividido em 4 packages principais:

### 6.1. Package: apresentacao

Aqui ficam as janelas da interface gráfica:

• Login.java: tela de entrada no sistema

• ConfortAnimal.java: janela principal com as três abas (Animal, Ambiente, Avaliações)

Aprendi que esta camada é responsável apenas por mostrar informação ao utilizador e recolher os dados que ele escreve. Não deve ter lógica de negócio aqui.

### 6.2. Package: model

Aqui ficam as classes que representam as entidades:

• Animal.java: classe pai com propriedades básicas

• Bovino.java: classe filha que herda de Animal

• Ambiente.java

• Avaliacao.java

• Usuario.java

Estas classes têm apenas getters, setters e métodos de validação. O Bovino tem um método validar() especial que verifica se todos os campos estão corretos antes de inserir na base de dados.

## 6.3. Package: service

Esta é a camada do meio, que tem a lógica de negócio:

• AnimalService.java

• AmbienteService.java

• AvaliacaoService.java: aqui fica o cálculo do ITU!

• BovinoService.java

• UsuarioService.java

Por exemplo, quando o utilizador clica em 'Registar Animal', a janela chama o AnimalService, que valida os dados e depois chama o AnimalDB para guardar.

## 6.4. Package: persistence

Esta camada faz a comunicação com a base de dados MySQL usando JDBC:

• Conexao.java: abre a ligação com o MySQL

• AnimalDB.java: INSERT, SELECT, DELETE de animais

• AmbienteDB.java

• AvaliacaoDB.java

• BovinoDB.java

• UsuarioDB.java

Aprendi que cada classe DB tem métodos como inserir(), listarTodos(), buscarPorId() e apagar(). Foi a primeira vez que usei PreparedStatement para evitar SQL Injection.

### 6.5. Package: exception

Criei exceções personalizadas para tratar erros de forma mais clara:

• ProjetoException: exceção base do projeto

• DadosInvalidosException: quando o utilizador insere dados errados

• ConexaoBDException: quando há problemas com a base de dados

No início usava apenas Exception, mas o professor explicou que é melhor ter exceções específicas para cada tipo de erro.

## 7. Conceitos de Programação Orientada a Objetos

Durante o desenvolvimento deste projeto, consegui aplicar vários conceitos importantes de POO que aprendemos na formação:

### 7.1. Herança

Usei herança na relação Animal → Bovino. O Bovino herda as propriedades nome, peso e idade do Animal, e adiciona raça, linhagem e produção de leite. Isto evita repetir código e organiza melhor as classes. Usei a palavra-chave 'extends' para criar a herança e 'protected' nas propriedades do Animal para que o Bovino possa aceder a elas no método validar().

### 7.2. Encapsulamento

Todos os atributos das minhas classes são privados (private) e só podem ser acedidos através de getters e setters. Por exemplo, não se pode fazer bovino.peso = -10 diretamente. Tem que usar o setPeso(), e este método valida se o peso é maior que zero antes de atribuir. Isto protege os dados e garante que só valores válidos são guardados.

### 7.3. Polimorfismo

Usei @Override no método toString() das classes para personalizar como os objetos são apresentados. Por exemplo, quando faço System.out.println(bovino), ele mostra as informações formatadas em vez do endereço de memória. Também poderia ter uma ArrayList<Animal> que na verdade contém objetos Bovino, graças ao polimorfismo.

### 7.4. Validação de Dados

Implementei validação em várias camadas:

1. Na interface gráfica: JOptionPane mostra mensagens de erro

2. Nos setters: lançam DadosInvalidosException se os dados forem inválidos

3. No método validar(): verifica tudo antes de inserir na base de dados

Isto garante que dados errados nunca chegam ao MySQL.

# 8. Base de Dados

Uso uma base de dados MySQL 8.0 que está num servidor remoto (62.28.39.135:3306). A base chama-se efa0125_08_projeto_java. O sistema tem 5 tabelas:

• usuario: guarda os utilizadores do sistema

• animal: tabela pai com dados básicos

• bovino: tabela filha com dados específicos de bovinos

• ambiente: guarda condições climáticas

• avaliacao: junta bovinos com ambientes e guarda resultados

Aprendi a usar chaves estrangeiras (Foreign Keys) para ligar as tabelas. Por exemplo:

• bovino.animal_id → animal.id (relação 1:1 de herança)

• avaliacao.bovino_id → bovino.id (relação N:1)

• avaliacao.ambiente_id → ambiente.id (relação N:1)

Usei CASCADE nas chaves estrangeiras. Isto significa que quando apago um animal, o MySQL apaga automaticamente o bovino correspondente e todas as avaliações daquele bovino. Isto evita ficar com dados órfãos na base de dados. No início tive dificuldades com o CASCADE porque não percebia bem como funcionava, mas depois de testar várias vezes consegui entender.

# 9. Dificuldades Encontradas

Encontrei várias dificuldades durante o desenvolvimento:

1. Herança com duas tabelas: **Foi difícil perceber como implementar a herança na base de dados. Demorei algum tempo a entender que precisava de duas tabelas separadas ligadas por chave estrangeira, e que tinha de usar transações para garantir que os INSERT em ambas as tabelas aconteciam ou falhavam juntos.**

2. PROTECTED vs PRIVATE: **No início não percebia porque é que o método validar() do Bovino não conseguia aceder aos campos do Animal. O professor explicou que tinha de usar PROTECTED em vez de PRIVATE para permitir acesso nas classes filhas.**

3. JTable e DefaultTableModel: **Tive muitas dificuldades em perceber como preencher as tabelas na interface. Aprendi que tinha de usar DefaultTableModel, limpar as linhas antigas com setRowCount(0) e adicionar novas linhas com addRow().**

4. Exceções: **No início não sabia quando usar try-catch. Aprendi que todas as operações com a base de dados podem falhar, por isso sempre tenho de ter try-catch e mostrar mensagens claras ao utilizador.**

5. CardLayout: **Para fazer a navegação entre as abas usei CardLayout, mas demorei a perceber como funcionava. Tinha que dar nomes aos 'cards' e usar show() para mudar entre eles.**

Apesar das dificuldades, consegui superar todas com a ajuda do professor, pesquisa na internet e muita tentativa e erro. Aprendi que programar é isso mesmo: tentar, falhar, perceber o erro e tentar de novo.

# 10. Conclusão

Este projeto foi uma experiência de aprendizagem muito enriquecedora. Consegui aplicar praticamente tudo o que aprendi na UFCD de Java Avançado:

✓ Programação Orientada a Objetos (herança, encapsulamento, polimorfismo)

✓ Interface gráfica com Swing

✓ Conexão com base de dados MySQL usando JDBC

✓ Arquitetura em camadas (apresentação, service, persistence)

✓ Exceções personalizadas

✓ Validação de dados

✓ Trabalho com chaves estrangeiras e CASCADE

**Mais importante:** Consegui criar algo que tem utilidade no mundo real. O Confort-Animal pode realmente ajudar produtores a melhorar o bem-estar dos seus animais e aumentar a produtividade das propriedades. Aprendi que programação não é só escrever código - é resolver problemas reais de forma organizada e pensada. Cada decisão que tomei (usar herança, separar em camadas, validar dados) teve um propósito e tornou o sistema melhor. Estou orgulhoso do resultado final e sinto que evoluí muito como programador durante este projeto. Claro que ainda tenho muito a aprender, mas este trabalho deu-me confiança para continuar a estudar e desenvolver sistemas cada vez melhores. Agradeço ao professor Rui Boticas pela orientação e por todos os ensinamentos durante a formação.

# O que tem no Projeto:

## TABELAS NO MYSQL:

1. **usuario**
2. **animal ← PAI**
3. **bovino ← FILHO (referencia animal)**
4. **ambiente**
5. **avaliacao**

## ENTIDADES NO JAVA:

1. **Usuario**
2. **Animal ← classe PAI**
3. **Bovino ← classe FILHA (extends Animal)**
4. **Ambiente**
5. **Avaliacao**

## Estrutura:

- **model/ - 5 classes simples (herança (Animal - Bovinos))**
- **persistence/ - Conexão + 5 DAOs básicos**
- **Exception/ - ProjetoException + ConexaoDBException + DadosInvalido-sEXception**
- **apresentation/ - LoginFrame + MainFrame(ConfortAnimal)**

## Login:

- **admin / admin**
- **lucas / lucas**

## Logout

- **Sair**

# Implementar na Aplicação:

## Aba Animais:

- **Propriedades: nome, peso, idade, raça(bovino), linhagem (bovino), producaoLeite (bovino).**

- **Funcionalidades: Clica em "Registar" Animal**
- **Consulta lista de animais em "Listar"**
- **Permite apagar animal e sua respectiva avaliação no banco de dados em "Apagar"**
- **selecionar animal para avaliação e armazenar histórico de avaliações por animal.**

## Aba Ambientes:

- **Propriedades: temperatura, umidade, local e dataRegisto.**

- **Funcionalidades: inserir dados climáticos em "Registar"**
- **atualizar dados climáticos por ambiente, consultar histórico de condições ambientais no "Listar"**
- **Permite apagar ambiente e sua respectiva avaliação no banco de dados em "Apagar"**
- **associar ambiente à avaliação.**

# Aba Avaliações:

- **Propriedades: resultado, valorITU e dataAvaliacao**
- **Gerar novas avaliações dos animais todos para o determinado ambiente em "Gerar Avaliação"**
- **Listar todas as avaliações novas e as que já existiam no banco de dados em "Listar Avaliação"**
- **Calcular e Listar automáticamente as Avalições de ITU**
- **Demonstrando por fim, se Animal esta em Conforto, Alerta ou Estresse Térmico.**